CloudLedger DevOps Capstone - 2-Minute Demo Video Script

Presenter: Utkarsh Raj | Target Duration: 2:00 Minutes | Modules Covered: M1 - M10 (100 Pts)

PRE-RECORDING CHECKLIST (Open These Before Hitting Record):

1. Browser Tab 1: CloudLedger UI (http://localhost:8080 or http://localhost:3000) with FinOps KPI cards visible.
2. Browser Tab 2: FastAPI Swagger UI (http://localhost:8000/docs) showing CRUD & /health endpoints.
3. Browser Tab 3: GitHub Actions (green CI/CD & Trivy 0 CVE logs) + GHCR Packages (40-char SHA tags).
4. Browser Tab 4: Prometheus Targets (http://localhost:9090/targets UP) + Grafana Dashboard (http://localhost:3001).
5. Terminal: Inside project root with Kind cluster running, ready for kubectl get pods/hpa & live patch commands.

0:00 - 0:25 (25s) | Modules 1, 2 & 4: Full-Stack App, Pytest & Docker Hardening

SHOW ON SCREEN: Start on Browser Tab 1 (CloudLedger Dashboard). Filter resources or click '+ Add Cloud Resource'. Briefly switch to Tab 2 (Swagger /docs).

SAY OUT LOUD: "Hi everyone, I'm Utkarsh, and this is CloudLedger - a FinOps and Cloud Infrastructure Cost-Auditing SaaS platform built for my DevOps Capstone. It tracks multi-cloud spend across AWS, GCP, and Azure, flags idle zombie assets, and calculates optimization health. The frontend runs on React and Vite inside a multi-stage, non-root Nginx container, backed by FastAPI, PostgreSQL, and Alembic migrations, with 6 isolated Pytest suites running against in-memory SQLite."

0:25 - 0:50 (25s) | Modules 3, 5 & 6: Git Hygiene, GitHub Actions CI/CD & Trivy DevSecOps

SHOW ON SCREEN: Switch to Browser Tab 3 (GitHub Actions green workflow run). Click the Trivy step showing 0 HIGH / 0 CRITICAL CVEs, then show the GHCR Packages page with the Git SHA tag.

SAY OUT LOUD: "Every push to main triggers our GitHub Actions CI/CD pipeline. Job 1 runs our isolated Pytest suite and compiles the Vite production bundle. Job 2 builds our non-root Docker images and scans both with Aqua Trivy configured with exit-code 1 on HIGH or CRITICAL CVEs. After passing with zero vulnerabilities, images are pushed to GitHub Container Registry tagged strictly with the immutable Git commit SHA."

0:50 - 1:15 (25s) | Modules 7 & 8: Terraform IaC (ap-south-1) & Kubernetes / Helm

SHOW ON SCREEN: Switch to Terminal. Show 'terraform validate' success (or M7 slide), then run: 'helm list -n cloudledger' and 'kubectl get pods,svc,ingress -n cloudledger' showing all 5 pods 1/1 Running.

SAY OUT LOUD: "For Infrastructure as Code, our Terraform modules define a custom VPC, public and private subnets, NAT Gateway, and managed EKS cluster in ap-south-1, validated via terraform fmt and validate. For orchestration, our custom Helm chart deploys 2 backend replicas, 2 frontend replicas, a stateful PostgreSQL PVC, ClusterIP services, and NGINX Ingress routing '/' and '/api' inside the cloudledger namespace."

1:15 - 1:35 (20s) | Module 9: Prometheus & Grafana Observability

SHOW ON SCREEN: Switch to Browser Tab 4. Show Prometheus Targets (cloudledger-backend-monitor UP 2/2) and Grafana live panel graphing rate(http_requests_total[1m]).

SAY OUT LOUD: "For observability, FastAPI exposes Prometheus metrics at /metrics. Using kube-prometheus-stack and a custom ServiceMonitor, Prometheus scrapes both backend pods every 15 seconds, and Grafana visualizes live HTTP request rates and latency percentiles in real time."

1:35 - 2:00 (25s) | Module 10: HPA Autoscaling & Live Kubernetes Troubleshooting

SHOW ON SCREEN: Switch to Terminal. Show 'kubectl get hpa -n cloudledger', then run the live kubectl set image / kubectl patch svc commands showing broken endpoints (<none>) immediately populating with Pod IPs.

SAY OUT LOUD: "Finally, for Module 10, our HorizontalPodAutoscaler scales the backend from 2 to 5 replicas at 70% CPU. And for live troubleshooting, we diagnosed ImagePullBackOff on a broken deployment using kubectl set image, and fixed a zero-endpoint service selector mismatch live using kubectl patch svc, restoring pod endpoint routing immediately. Thank you!"